

- 1) In diverse esercitazioni è stato richiesto di leggere dei numeri da tastiera con uno Scanner. Se il dato inserito non è del formato corretto viene lanciata una `java.util.InputMismatchException`. Modificate il Gestore Schermo dell'esercitazione precedente in modo da gestire le eventuali eccezioni lanciate. Modificate quindi i costruttori delle classi per le figure geometriche per fare in modo che lancino una `IllegalArgumentException` nel caso in cui vengono passati dati errati (ad esempio raggio del cerchio negativo). Gestite le eventuali eccezioni lanciate aggiungendo gli opportuni `try/catch` nel `main` del `GestoreSchermo`.
- 2) Definire una classe **Stack** che implementi una pila di 100 String tramite una `LinkedList<String>`. Le funzioni membro della classe devono essere:

```
void push(String s)
String pop()
boolean isEmpty()
boolean isFull().
```

Scrivete un programma che crea un oggetto **Stack** e verifica il corretto funzionamento della classe. Implementate infine i metodi `toString` e `equals` e verificate il corretto funzionamento. Fate lanciare delle eccezioni se si prova a inserire un elemento nello stack pieno o a estrarre da uno stack vuoto.
- 3) (Progetto completo) Creare una gerarchia di classi che possa rappresentare le seguenti entità: **Persona**, **Professore**, **Studente**, **StudenteTriennale** e **StudenteMagistrale**.
Ogni **Professore** ha una `dataAssunzione`, un ruolo (Ricercatore, Professore Associato o Professore Ordinario), un dipartimento di appartenenza (es. DIID, DICAM, ...). Ogni Professore percepisce uno *stipendio* (prevedere quindi i metodi `getStipendio()` e `setStipendio()`).
Ogni **Studente** ha una `dataIscrizione`, una `matricola`, un `corsoDiLaurea` a cui è iscritto. Ogni **Studente** paga un contributo d'Iscrizione al corso.
Uno **StudenteTriennale** deve conseguire 180 CFU e proviene da una `scuolaSuperiore` (una stringa per memorizzare la scuola di provenienza). Uno **StudenteMagistrale** deve conseguire 120 CFU e proviene da un `corsoTriennale` (una stringa per memorizzare il corso di laurea triennale di provenienza).
Prevedere opportuni metodi per l'incapsulamento dei dati.
Si scriva quindi una classe per la gestione del corso di laurea. Tale classe utilizzerà un'unica collezione eterogenea per memorizzare i professori e gli studenti del corso (al massimo 100). Prevedere un metodo che consenta di stampare il costo totale derivante dal pagamento dei salari dei professori (cioè la somma dei salari dei professori) ed un metodo che consenta di stampare il ricavo proveniente dai contributi d'iscrizione versati dagli studenti.
Gestite eventuali situazioni anomale tramite opportune eccezioni.

ESERCIZI DEL LIBRO DI TESTO

Esercizio 9.7 Niente multipli di 7

Creare un'eccezione checked chiamata `MultiploDi7Exception` che dovrebbe essere lanciata quando un numero coinvolto in un'operazione è multiplo di 7. Inserire all'interno metodi e attributi che si ritengono opportuni.

Esercizio 9.18 Gestore Divisioni

Partendo dalla soluzione dell'esercizio 9.7:

1. Creare una classe chiamata `Calcolatore` che contiene un metodo per eseguire la divisione. Se il divisore è 7, lanciare l'eccezione personalizzata `MultiploDi7Exception`.
2. Creare una classe chiamata `GestoreDivisioni`, che chiama il metodo della classe `Calcolatore` e gestisce l'eccezione `MultiploDi7Exception`.

Esercizio 9.20 Gestore File

1. Creare una classe chiamata `FileReaderRisorsa` con un metodo `leggiDati` che implementa l'interfaccia `AutoCloseable`. Questa classe simulerà la lettura di un file e, quando la risorsa viene chiusa, dovrebbe stampare un messaggio di chiusura.
2. Creare una classe `GestoreFile` che utilizza il costrutto `try-with-resources` per gestire l'istanza di `FileReaderRisorsa` e simula la lettura di un file all'interno del blocco `try`.